



Ejercicios de Programación

en JavaScript

Guía práctica para principiantes

Temas cubiertos:

Seleccionar elementos · Modificar texto y estilos
Crear elementos · Insertar y eliminar nodos · Eventos

Progresión de dificultad

Muy Fácil



Fácil



Medio

Índice

Antes de comenzar	2
1. Nivel Muy Fácil — Seleccionar y modificar elementos	3
2. Nivel Fácil — Eventos y lectura de valores	6
3. Nivel Medio — Crear, insertar y eliminar nodos	9
Referencia rápida	16

Antes de comenzar

Esta guía contiene ejercicios progresivos para practicar la **manipulación del DOM con JavaScript**. Cada ejercicio incluye el HTML base ya listo; vos solo tenés que escribir el JavaScript.

💡 **Consejo:** intentá resolver cada ejercicio antes de leer la pista. Si el resultado no es el esperado, usá la consola del navegador (F12) para ver los errores. *Equivocarse es parte del proceso.*

¿Dónde probar el código?

- Creá un archivo `index.html` y abrílo en el navegador.
- O usá editores online como **CodePen** o **JSFiddle** (pegá el HTML en el panel HTML y el JS en el panel JS).

Nivel	Qué practicás
Muy Fácil	Seleccionar un elemento y modificar su texto o estilo.
Fácil	Reaccionar a eventos de usuario con <code>addEventListener</code> .
Medio	Crear, insertar y eliminar nodos del DOM dinámicamente.

Nivel Muy Fácil — Seleccionar y modificar elementos

i ¿Qué es el DOM? El DOM (*Document Object Model*) es la representación en memoria que hace el navegador de tu página HTML. Con JavaScript podés acceder a cada elemento y cambiar su contenido, estilos o atributos **sin recargar la página**.

```
1 // Seleccionar por id
2 const titulo = document.getElementById("miTitulo");
3
4 // Seleccionar con selector CSS (devuelve el primero que coincide)
5 const parrafo = document.querySelector(".descripcion");
6
7 // Modificar el texto visible
8 titulo.textContent = "Nuevo título";
9
10 // Cambiar un estilo directamente
11 titulo.style.color = "blue";
12
13 // Agregar o quitar una clase CSS
14 titulo.classList.add("resaltado");
15 titulo.classList.remove("resaltado");
```

</> ☆ Ejercicio 1 • Cambiar el título de la página

Muy Fácil

getElementById

textContent

Enunciado

La página tiene un título que dice “*Esperando cambio...*”. Cuando el usuario haga clic en el botón, el título debe cambiar a “*¡Bienvenido al DOM!*” y el color del texto debe volverse azul marino.

HTML base

```
1 <h1 id="titulo">Esperando cambio...</h1>
2 <button id="btnCambiar">Cambiar titulo</button>
3
4 <script src="ej1.js"></script>
```

Resultado esperado

Al hacer clic: el h1 muestra “**¡Bienvenido al DOM!**” en color azul marino (navy).

Código inicial:

```
1  const titulo      = document.getElementById("titulo");
2  const btnCambiar = document.getElementById("btnCambiar");
3
4  btnCambiar.addEventListener("click", function() {
5      // 1. Cambiá el textContent del título
6      // 2. Cambiá el color del título a "navy"
7  });
```

💡 **Pista:** Para cambiar el texto usá `titulo.textContent = "..."`. Para el color usá `titulo.style.color = "navy"`. Ambas líneas van dentro del bloque `function() { }`.

🔗 Ejercicio 2 • Semáforo de colores

Muy Fácil

style

classList

Enunciado

Hay un cuadrado en pantalla y tres botones: **Rojo**, **Amarillo** y **Verde**. Al hacer clic en cada botón, el cuadrado debe cambiar al color correspondiente.

HTML base

```
1  <div id="cuadrado"
2      style="width:120px; height:120px; background:gray;
3          border-radius:8px; margin-bottom:12px;">
4  </div>
5
6  <button id="btnRojo">Rojo</button>
7  <button id="btnAmarillo">Amarillo</button>
8  <button id="btnVerde">Verde</button>
9
10 <script src="ej2.js"></script>
```

Código inicial:

```
1  const cuadrado    = document.getElementById("cuadrado");
2  const btnRojo     = document.getElementById("btnRojo");
3  const btnAmarillo = document.getElementById("btnAmarillo")
4  ;
5  const btnVerde    = document.getElementById("btnVerde");
6  // Asigná un evento a cada botón para cambiar el color del
   // cuadrado
```

💡 **Pista:** Usa `cuadrado.style.backgroundColor = "red"` dentro del `addEventListener` de cada botón. Los valores CSS son: "red", "yellow", "green".

Nivel Fácil — Eventos y lectura de valores

i Leer el valor de un input: Para obtener lo que el usuario escribe en un campo de texto usamos la propiedad `.value`

```
1  const campo = document.getElementById("miInput");
2
3  // Al hacer clic en un botón, leer lo que escribió el
   usuario
4  boton.addEventListener("click", function() {
5      const texto = campo.value;           // lee el texto del
      input
6      console.log("Escribiste:", texto);
7      campo.value = "";                   // limpia el campo
8  });
```

</> Ejercicio 3 • Formulario de Login

Fácil`.value``textContent``style`

Enunciado

Implementá la validación de un formulario de login **sin enviar datos a ningún servidor**:

- Si el usuario o la contraseña están vacíos, mostrar *“Completá todos los campos.”* en rojo.
- Si el usuario es `admin` y la contraseña es `1234`, mostrar *“¡Bienvenido, admin!”* en verde.
- En cualquier otro caso, mostrar *“Credenciales incorrectas.”* en rojo.
- Cada vez que el usuario empiece a escribir de nuevo, el mensaje debe desaparecer.

HTML base

```
1 <h2>Iniciar sesion</h2>
2
3 <label>Usuario:</label>
4 <input type="text" id="usuario" placeholder="Tu
   usuario" /><br><br>
5
6 <label>Contrasena:</label>
7 <input type="password" id="contrasena" placeholder="Tu
   contrasena" /><br><br>
8
9 <button id="btnLogin">Ingresar</button>
10
11 <p id="mensaje" style="font-weight:bold;"></p>
12
13 <script src="ej3.js"></script>
```

Código inicial:

```
1 const inputUsuario = document.getElementById("usuario");
2 const inputContrasena = document.getElementById("
   contrasena");
3 const btnLogin      = document.getElementById("btnLogin"
   );
4 const mensaje       = document.getElementById("mensaje")
   ;
5
6 // Limpiar el mensaje cuando el usuario empieza a escribir
7 inputUsuario.addEventListener("input", function() {
8   // borrar el texto del mensaje
9 });
10
11 inputContrasena.addEventListener("input", function() {
12   // borrar el texto del mensaje
13 });
14
15 btnLogin.addEventListener("click", function() {
16   const usuario = inputUsuario.value.trim();
17   const contrasena = inputContrasena.value.trim();
18
19   // Validar campos vacíos
20   // Validar usuario y contraseña correctos
21   // Mostrar mensaje correspondiente con color
22 });
```

💡 **Pista:** Para mostrar el mensaje usá `mensaje.textContent = "..."` y para el color `mensaje.style.color = "red"` o `"green"`. Para borrar el mensaje ponelos en vacío: `mensaje.textContent = ""`. Verificá el campo vacío con `usuario === ""`.

🔗 Ejercicio 4 • Mostrar nombre en tiempo real

Fácil

evento input

.value

textContent

Enunciado

Hay un campo de texto y un párrafo de vista previa. **A medida que el usuario escribe su nombre**, el párrafo debe actualizarse instantáneamente mostrando: *"Holá, [nombre]!"*. Si el campo está vacío, mostrar *"Escribí tu nombre arriba."*

HTML base

```
1 <label>Tu nombre:</label>
2 <input type="text" id="inputNombre" placeholder="Escribi
  aqui..." />
3
4 <p id="preview">Escribi tu nombre arriba.</p>
5
6 <script src="ej4.js"></script>
```

Código inicial:

```
1 const inputNombre = document.getElementById("inputNombre")
  ;
2 const preview      = document.getElementById("preview");
3
4 // Escuchá el evento "input" del campo de texto
5 inputNombre.addEventListener("input", function() {
6   // Si el campo está vacío      mostrar "Escribi tu nombre
   arriba."
7   // Si no                      mostrar "Holá, [nombre]!"
8 });
```

❗ **Evento "input" vs "change"**: "input" se dispara **en cada tecla** que el usuario presiona, por eso sirve para vista previa en tiempo real. "change" se dispara solo cuando el campo pierde el foco, lo que no daría la sensación de tiempo real.

Nivel Medio — Crear, insertar y eliminar nodos

i Crear e insertar elementos: Con JavaScript podés fabricar nodos nuevos y agregarlos al DOM sin necesidad de tocar el HTML.

```
1 // 1. Crear el elemento
2 const nuevoParrafo = document.createElement("p");
3
4 // 2. Darle contenido
5 nuevoParrafo.textContent = "Soy un párrafo nuevo";
6
7 // 3. Insertarlo en el DOM (al final de un contenedor)
8 const contenedor = document.getElementById("contenedor");
9 contenedor.appendChild(nuevoParrafo);
10
11 // 4. Eliminar un elemento
12 nuevoParrafo.remove();
```

</> ≡ Ejercicio 5 • Lista de invitados

Medio

createElement

appendChild

remove

Enunciado

Construí una lista de invitados dinámica:

- Al escribir un nombre y hacer clic en **Agregar**, se debe crear un `<li>` con ese nombre y agregarlo a la lista.
- No se pueden agregar nombres vacíos (mostrar un mensaje de aviso y no agregar nada).
- Cada `<li>` debe tener un botón **Quitar** que, al hacer clic, elimina **solo ese elemento** de la lista.
- Después de agregar, el campo de texto debe quedar vacío y con el foco.

HTML base

```
1 <h2>Lista de invitados</h2>
2
3 <input type="text" id="inputNombre" placeholder="Nombre
  del invitado" />
4 <button id="btnAgregar">Agregar</button>
5 <p id="aviso" style="color:red;"></p>
6
7 <ul id="listaInvitados"></ul>
8
9 <script src="ej5.js"></script>
```

Código inicial:

```
1  const inputNombre      = document.getElementById("
    inputNombre");
2  const btnAgregar       = document.getElementById("btnAgregar
    ");
3  const listaInvitados   = document.getElementById("
    listaInvitados");
4  const aviso            = document.getElementById("aviso");
5
6  btnAgregar.addEventListener("click", function() {
7      const nombre = inputNombre.value.trim();
8
9      // Si el nombre está vacío: mostrar aviso y salir
10     if (nombre === "") {
11         // completá
12         return;
13     }
14
15     // Ocultar el aviso si había uno
16     aviso.textContent = "";
17
18     // 1. Crear el <li>
19     const li = document.createElement("li");
20
21     // 2. Poner el nombre como texto del <li>
22     li.textContent = nombre;
23
24     // 3. Crear el botón "Quitar"
25     const btnQuitar = document.createElement("button");
26     btnQuitar.textContent = "Quitar";
27
28     // 4. Al hacer clic en "Quitar", eliminar el <li>
29     btnQuitar.addEventListener("click", function() {
30         // completá
31     });
32
33     // 5. Insertar el botón dentro del <li>
34     li.appendChild(btnQuitar);
35
36     // 6. Insertar el <li> dentro de la lista
37     listaInvitados.appendChild(li);
38
39     // 7. Limpiar y enfocar el campo
40     inputNombre.value = "";
41     inputNombre.focus();
42 });
```

💡 **Pista:** Para el botón Quitar, dentro de su `addEventListener` escribía simplemente `li.remove()` : como `li` fue declarado en el mismo scope, el botón puede accederlo directamente. Para mostrar el aviso usá `aviso.textContent = "Ingresá un nombre."`.

</> Ejercicio 6 • Alternar contenido con un botón

Medio

classList.toggle

textContent

setAttribute

Enunciado

Hay una tarjeta con información y un botón que dice “**Ocultar**”. Al hacer clic:

- Si la tarjeta está visible, ocultarla y cambiar el texto del botón a “**Mostrar**”.
- Si la tarjeta está oculta, mostrarla y cambiar el texto del botón a “**Ocultar**”.

Para ocultar/mostrar, **no modifiques el estilo directamente**: usá `classList.toggle` con una clase CSS llamada `oculto`.

HTML base

```
1 <style>
2   .oculto { display: none; }
3   .tarjeta { background: #f0f0f0; padding: 16px;
4             border-radius: 8px; margin-bottom: 12px; }
5 </style>
6
7 <div class="tarjeta" id="tarjeta">
8   <h3>Informacion importante</h3>
9   <p>Este contenido puede mostrarse u ocultarse con el
10     boton.</p>
11 </div>
12 <button id="btnToggle">Ocultar</button>
13
14 <script src="ej6.js"></script>
```

Código inicial:

```
1 const tarjeta = document.getElementById("tarjeta");
2 const btnToggle = document.getElementById("btnToggle");
3
4 btnToggle.addEventListener("click", function() {
5   // 1. Alternar la clase "oculto" en la tarjeta
6   // 2. Cambiar el texto del botón según el estado
7 });
```

i `classList.toggle("clase")` : si el elemento **tiene** la clase, la quita; si **no la tiene**, la agrega. Es la forma correcta de alternar estados en el DOM. Retorna `true` si la clase quedó agregada, `false` si fue quitada, lo que sirve para saber en qué estado quedó el elemento.

</> ☰ Ejercicio 7 • Generador de tarjetas de perfil

Medio

createElement

appendChild

setAttribute

Enunciado

El usuario completa un formulario con **nombre**, **profesión** y **color favorito**. Al hacer clic en **Generar tarjeta**, se debe crear dinámicamente una tarjeta en la sección de resultados con:

- Un `<h3>` con el nombre.
- Un `<p>` con la profesión.
- El borde izquierdo de la tarjeta del color elegido (usá `style.borderLeft`).
- Un botón **Eliminar tarjeta** que borra esa tarjeta.

No se puede generar una tarjeta si el nombre o la profesión están vacíos.

HTML base

```
1 <h2>Crear perfil</h2>
2 <input type="text" id="inpNombre" placeholder="Nombre"
  /><br>
3 <input type="text" id="inpProfesion" placeholder="
  Profesion" /><br>
4 <input type="color" id="inpColor" value="#3b82f6"
  /><br>
5 <button id="btnGenerar">Generar tarjeta</button>
6 <p id="avisoP" style="color:red;"></p>
7
8 <div id="resultados"></div>
9
10 <script src="ej7.js"></script>
```

Código inicial:

```
1  const inpNombre      = document.getElementById("inpNombre");
2  const inpProfesion   = document.getElementById("inpProfesion");
3  const inpColor       = document.getElementById("inpColor");
4  const btnGenerar     = document.getElementById("btnGenerar");
5  const resultados     = document.getElementById("resultados");
6  const avisoP         = document.getElementById("avisoP");
7
8  btnGenerar.addEventListener("click", function() {
9      const nombre      = inpNombre.value.trim();
10     const profesion    = inpProfesion.value.trim();
11     const color        = inpColor.value;
12
13     // Validar que nombre y profesión no estén vacíos
14
15     // Crear la tarjeta: un <div> con:
16     //   - style.border      = "4px solid " + color
17     //   - style.borderRadius = "8px"
18     //   - style.padding     = "12px"
19     //   - style.marginTop   = "10px"
20
21     // Crear el <h3> con el nombre
22
23     // Crear el <p> con la profesión
24
25     // Crear el botón "Eliminar tarjeta"
26
27     // Armar la jerarquía y agregar al DOM
28 });
```

💡 **Pista:** El orden de `appendChild` importa: primero agregá el `h3`, después el `p` y por último el botón al `div` de la tarjeta. Para el borde usá: `tarjeta.style.borderLeft = "6px solid " + color`.

Referencia rápida

Seleccionar elementos

```
1 document.getElementById("miId")           // por id (único)
2 document.querySelector(".miClase")        // primer elemento con
   esa clase
3 document.querySelector("h2")              // primer <h2> del
   documento
4 document.querySelectorAll("li")           // todos los <li> =>
   NodeList
```

Modificar contenido y estilos

```
1 elemento.textContent = "Nuevo texto";     // cambia el texto
   visible
2 elemento.innerHTML   = "<b>Negrita</b>";    // permite HTML
   interno
3 elemento.style.color = "red";              // estilo en línea
4 elemento.style.fontSize = "20px";
5 elemento.classList.add("activo");          // agrega clase CSS
6 elemento.classList.remove("activo");       // quita clase CSS
7 elemento.classList.toggle("activo");       // alterna clase
   CSS
8 elemento.setAttribute("src", "foto.jpg"); // asigna atributo
```

Crear, insertar y eliminar

```
1 const nuevoDiv = document.createElement("div"); // crear
2 nuevoDiv.textContent = "Hola";                  // dar
   contenido
3 contenedor.appendChild(nuevoDiv);              // insertar al
   final
4 nuevoDiv.remove();                              // eliminar del
   DOM
```

Eventos comunes

Evento	Cuándo se dispara
"click"	Al hacer clic sobre el elemento.
"input"	Cada vez que el valor del campo cambia (en tiempo real).
"change"	Cuando el campo pierde el foco con un valor distinto.
"keydown"	Al presionar una tecla; <code>event.key</code> indica cuál.

Errores comunes

Error	Causa y solución
<code>Cannot read properties of null</code>	El elemento no existe o el <code><script></code> está antes del HTML. Mové el <code><script></code> al final del <code><body></code> .
El evento no responde	Verificá que el <code>id</code> del HTML coincida exactamente con el usado en <code>getElementById</code> .
<code>textContent</code> no cambia	Asegurate de estar modificando el elemento correcto y que el listener esté bien registrado.



¡Buena suerte con el DOM!